



QUESTIONS

8 / 8 completed

Join

JOIN

Write a Java program to demonstrate how to use the "Join" method to wait for a thread to complete its execution before proceeding with the main thread.

PROGRAM EDITOR

Java

Run

Save

```
1 class Main {
2     static class MyThread extends Thread {
3         public void run() {
4             for (int i = 1; i <= 5; i++) {
5                 System.out.println("Thread: " + i);
6             }
7         }
8     }
9
10    public static void main(String[] args) throws Exception {
11        MyThread t = new MyThread();
12
13        t.start();
14
15        t.join();
16
17        System.out.println("Main thread completed");
18    }
19 }
```

OUTPUT

```
Thread: 1
Thread: 2
Thread: 3
Thread: 4
Thread: 5
Main thread completed
```

INPUT

Your INPUT goes here!



QUESTIONS

8 / 8 completed

M-Thread

M-THREAD

Write a Java program to create multiple threads that perform some time-consuming task, such as calculating prime numbers or sorting an array, and compare their execution times.

PROGRAM EDITOR

Java

Run

Save

```
1 class Main {
2     static class Task extends Thread {
3         private int number;
4
5         Task(int number) {
6             this.number = number;
7         }
8
9         public void run() {
10             for (int i = 1; i <= 5; i++) {
11                 System.out.println("Thread " + number + ": " + i);
12             }
13         }
14     }
15
16     public static void main(String[] args) throws Exception {
17         Thread t1 = new Task(1);
18         Thread t2 = new Task(2);
19         Thread t3 = new Task(3);
20
21         t1.start();
```

OUTPUT

```
Thread 3: 1
Thread 2: 1
Thread 2: 2
Thread 1: 1
Thread 2: 3
Thread 3: 2
Thread 2: 4
Thread 1: 2
Thread 2: 5
Thread 3: 3
```

INPUT

Your INPUT goes here!



QUESTIONS

8 / 8 completed

1-10 T

1-10 T

Write a Java program to create a simple thread that prints the numbers from 1 to 10.

PROGRAM EDITOR

Java

Run

Save

```
1 class Main {
2     public static void main(String[] args) {
3         Thread t = new Thread() -> {
4             for (int i = 1; i <= 10; i++) {
5                 System.out.println(i);
6             }
7         });
8
9         t.start();
10
11        try {
12            t.join();
13        } catch (InterruptedException e) {
14            System.out.println("Interrupted");
15        }
16    }
17 }
```

OUTPUT

```
1
2
3
4
5
6
7
8
9
10
```

INPUT

Your INPUT goes here!



QUESTIONS

8 / 8 completed

Pool

POOL

Write a Java program to create a thread pool that can execute multiple threads concurrently.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.concurrent.ExecutorService;
2 import java.util.concurrent.Executors;
3
4 class Main {
5     public static void main(String[] args) throws Exception {
6         ExecutorService pool = Executors.newFixedThreadPool(3);
7
8         for (int i = 1; i <= 5; i++) {
9             final int task = i;
10
11             pool.execute(() -> {
12                 System.out.println("Task " + task + " executed");
13             });
14         }
15
16         pool.shutdown();
17
18         while (!pool.isTerminated()) {
19             Thread.sleep(10);
20         }
21 }
```

OUTPUT

```
Task 3 executed
Task 1 executed
Task 2 executed
Task 5 executed
Task 4 executed
All tasks completed
```

INPUT

Your INPUT goes here!



QUESTIONS

8 / 8 completed

Counter-T

COUNTER-T

Write a Java program to create a thread-safe counter that can be accessed by multiple threads simultaneously.

PROGRAM EDITOR

Java

Run

Save

```
1 class Main {  
2     static class Counter {  
3         private int count = 0;  
4  
5         synchronized void increment() {  
6             count++;  
7         }  
8  
9         int getCount() {  
10            return count;  
11        }  
12    }  
13  
14    public static void main(String[] args) throws Exception {  
15        Counter c = new Counter();  
16  
17        Thread t1 = new Thread(() -> {  
18            for (int i = 0; i < 1000; i++)  
19                c.increment();  
20        });  
21    }
```

OUTPUT

Counter: 2000

INPUT

Your INPUT goes here!



QUESTIONS

8 / 8 completed

Pro-Consume

PRO-CONSUME

Write a Java program to create a producer-consumer scenario using multiple threads. The producer thread should add items to a queue, and the consumer thread should remove items from the queue and process them.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.LinkedList;
2 import java.util.Queue;
3
4 class Main {
5     static Queue<Integer> queue = new LinkedList<>();
6
7     static class Producer extends Thread {
8         public void run() {
9             for (int i = 1; i <= 5; i++) {
10                 synchronized (queue) {
11                     queue.add(i);
12                     System.out.println("Produced: " + i);
13                     queue.notify();
14                 }
15             }
16         }
17     }
18
19     static class Consumer extends Thread {
20         public void run() {
21             for (int i = 1; i <= 5; i++) {
```

OUTPUT

```
Produced: 1
Produced: 2
Produced: 3
Produced: 4
Produced: 5
Consumed: 1
Consumed: 2
Consumed: 3
Consumed: 4
Consumed: 5
```

INPUT

Your INPUT goes here



QUESTIONS

8 / 8 completed

Wait-Notify

WAIT-NOTIFY

Write a Java program to demonstrate how to use the "wait" and "notify" methods to implement a producer-consumer scenario.

PROGRAM EDITOR

Java

Run

Save

```
1 class Main {
2     static int value = 0;
3     static boolean available = false;
4
5     static class Producer extends Thread {
6         public void run() {
7             synchronized (Main.class) {
8                 value = 100;
9                 available = true;
10                System.out.println("Produced: " + value);
11                Main.class.notify();
12            }
13        }
14    }
15
16    static class Consumer extends Thread {
17        public void run() {
18            synchronized (Main.class) {
19                while (!available) {
20                    try {
21                        Main.class.wait();
22                    } catch (InterruptedException e) {}
23                }
24            }
25        }
26    }
27 }
```

OUTPUT

```
Produced: 100
Consumed: 100
Process completed
```

INPUT

Your INPUT goes here!



QUESTIONS

8 / 8 completed

Deadlock

DEADLOCK

Write a Java program to create a deadlock scenario using multiple threads.

PROGRAM EDITOR

Java

Run

Save

```
1 class Main {
2     static Object lock1 = new Object();
3     static Object lock2 = new Object();
4
5     static class ThreadOne extends Thread {
6         public void run() {
7             synchronized (lock1) {
8                 System.out.println("Thread 1 locked Resource 1");
9
10                synchronized (lock2) {
11                    System.out.println("Thread 1 locked Resource 2");
12                }
13            }
14        }
15    }
16
17    static class ThreadTwo extends Thread {
18        public void run() {
19            synchronized (lock2) {
20                System.out.println("Thread 2 locked Resource 2");
21            }
22        }
23    }
24 }
```

OUTPUT

```
Thread 1 locked Resource 1
Thread 1 locked Resource 2
Thread 2 locked Resource 2
Thread 2 locked Resource 1
Deadlock scenario demonstrated
```

INPUT

Your INPUT goes here!